

Team: Hybrid-LA

Der große Preis von Heidenheim, Programmieren 1,
Sommersemester 2025/26

Unser Team

Anton Moser



- Programmierung und Implementierung der Regelung
- Optimierung
- GitHub-Management
- Erstellung Readme/Doku

Lukas Mühlhofer



- Erstellung Sequenzdiagramm/Doku
- Optimierung
- Statistische Bestimmung der Einstellparameter
- PowerPoint-Präsentation

Team-Name: Hybrid → Anspielung auf den Ansatz bei unserem Algorithmus
 LA → L=Lukas, A=Anton; oder „läuft irgendwie auch“; oder „Lernen durch Ausprobieren“

Clean Code im Projekt

- Ergänzung von aussagekräftigen Kommentaren

```
def begrenzen(wert: int, minimum: int, maximum: int) -> int:  
    # Damit kein Motor mehr bekommt als er verkraften kann  
    return max(minimum, min(maximum, wert))
```

Effekt:

- Visualisierung des Codes auf eine andere Art und Weise

```
def begrenzen(wert: int, minimum: int, maximum: int) -> int:  
    # Damit er nicht auf der Stelle dreht oder übersteuert  
    return max(minimum, min(maximum, wert))
```

Clean Code im Projekt

- Magic Numbers wurden am Anfang des Programmcodes in aussagekräftige Parameternamen geändert

```
def nach_links_korrigieren(korrektur: int) -> None:
    # Links abbremsen, rechts Gas geben -> dreht nach links
    # Bei sehr großer Korrektur fährt links rückwärts. Features, keine Bugs.
    linker_motor = max(MINDEST_GESCHWINDIGKEIT_RUECKWAERTS, -GRUND_GESCHWINDIGKEIT - korrektur)
    rechter_motor = GRUND_GESCHWINDIGKEIT + korrektur
    motoren_setzen(linker_motor, rechter_motor)

def nach_rechts_korrigieren(korrektur: int) -> None:
    # Rechts abbremsen, links Gas geben -> dreht nach rechts
    linker_motor = GRUND_GESCHWINDIGKEIT + korrektur
    rechter_motor = max(MINDEST_GESCHWINDIGKEIT_RUECKWAERTS, -GRUND_GESCHWINDIGKEIT - korrektur)
    motoren_setzen(linker_motor, rechter_motor)
```

Clean Code im Projekt

- Symmetrischer Aufbau bei Abfragen
 - Erzeugt Wiedererkennungswert
 - Verbessert Lesbarkeit
 - Einfacher im Debugging
 - Unterscheidung unterstützt durch Kommentare

```
# — Mitte + links -> fast richtig, bisschen links -
elif mitte == 1 and links == 1 and rechts == 0:
    korrektur = KORREKTUR_GERADE
    nach_links_korrigieren(korrektur)

    letzte_position = -1
    letzte_mitte_zeit = jetzt
    aktuelle_kurvenrichtung = 0
    kurven_startzeit = None

    aktion = f"LEICHT LINKS | Korrektur={korrektur} "

# — Mitte + rechts -> fast richtig, bisschen rechts -
elif mitte == 1 and rechts == 1 and links == 0:
    korrektur = KORREKTUR_FAST_MITTE
    nach_rechts_korrigieren(korrektur)

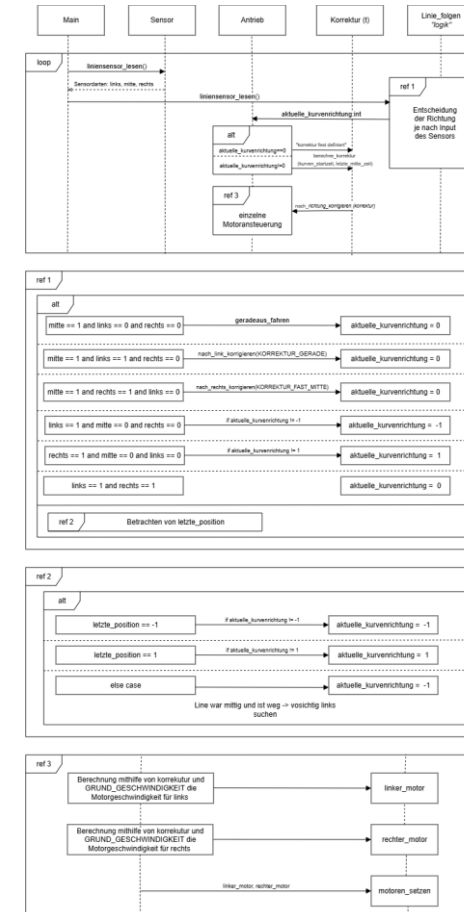
    letzte_position = 1
    letzte_mitte_zeit = jetzt
    aktuelle_kurvenrichtung = 0
    kurven_startzeit = None

    aktion = f"LEICHT RECHTS | Korrektur={korrektur}"
```

Algorithmus der Regelung

Übersicht:

- Sequenzdiagramm an dem in der Wirtschaft üblichen UML-Standard angelegt
- Unterteilung:
 - 1 Übersicht mit Lebenslinien
 - 3 Unterdiagramme zur genaueren Beschreibung in „ref“ Frames

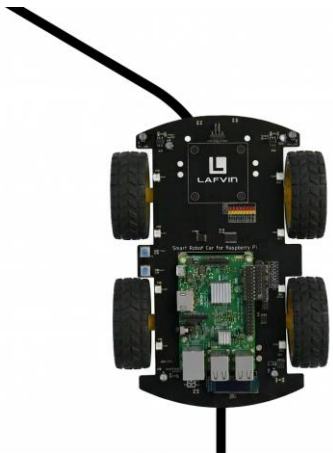


Algorithmus der Regelung

Zustände:

Links:

- Zeitlich linear Ansteigende Korrektur im Bereich
[0; KORREKTUR_MAX]



Halb Links:

- Hartkodierte Gegenlenkung



Gerade:

- Gleichmäßige Ansteuerung



Halb Rechts:

- Hartkodierte Gegenlenkung



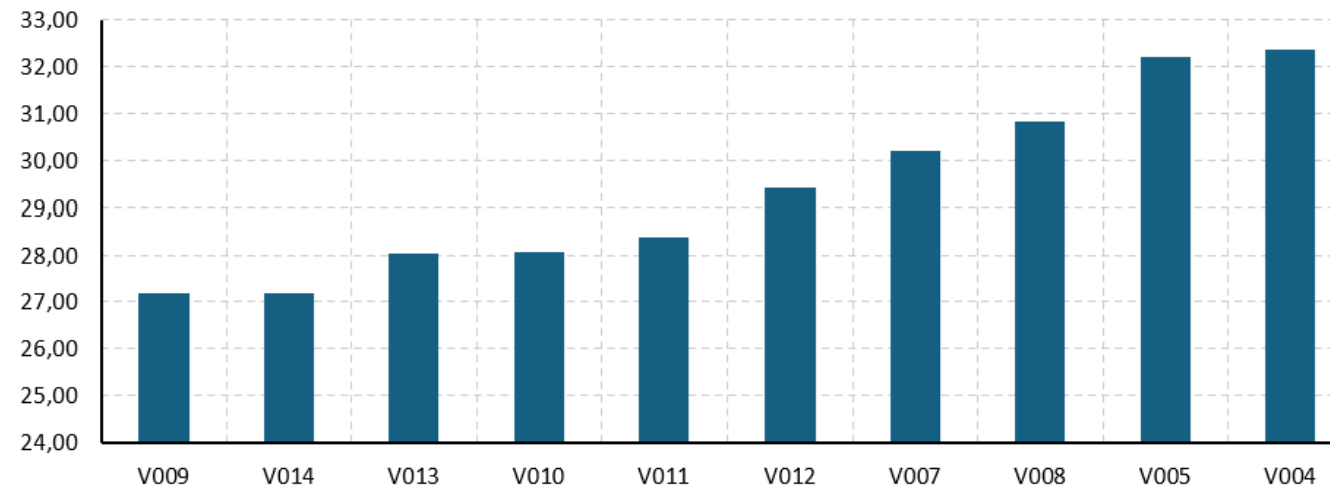
Rechts:

- Zeitlich linear Ansteigende Korrektur im Bereich
[0; KORREKTUR_MAX]



Empirische Ermittlung der Parameter

Top 10 – Mittelwert der Rundenzeit



Rang	Hilfswert	Test_ID	Mittelwert_s	StdAbw_s	Bestzeit_s	Grund_Geschwindigkeit	Min_Moto	Max_Moto	Losrhr_Geschwindigkeit	Korrektur_Gerade	Korrektur_Straße	Korrektur_M	Korrektur_Anstiege	Gerade_FoI
1	27,20013	V009	27,20	0,18	27,00	35	-25	45	empty	8	5	35	12	0.3
2	27,20018	V014	27,20	0,66	26,40	35	-25	45	empty	8	5	35	12	0.3
3	28,02517	V013	28,03	1,05	26,60	35	-25	45	empty	8	5	35	12	0.3
4	28,07514	V010	28,08	0,83	27,00	37	-25	42	empty	8	5	35	12	0.8
5	28,37515	V011	28,38	1,70	27,00	37	-25	42	empty	8	5	35	12	0.8
6	29,42516	V012	29,43	1,07	28,60	35	-25	45	empty	8	5	35	12	0.8
7	30,20011	V007	30,20	0,98	29,00	30	-25	45	empty	10	8	35	12	0.5
8	30,82512	V008	30,83	0,59	30,00	30	-25	45	empty	8	5	35	12	0.5
9	32,20009	V005	32,20	0,85	31,60	30	-22	40		20	10	8	35	12 0.15
10	32,37508	V004	32,38	0,45	32,00	30	-22	40		20	10	12	35	10 0.15

*nicht alle Parameter dargestellt, für alle vgl. GitHub

Vorteile des Algorithmus

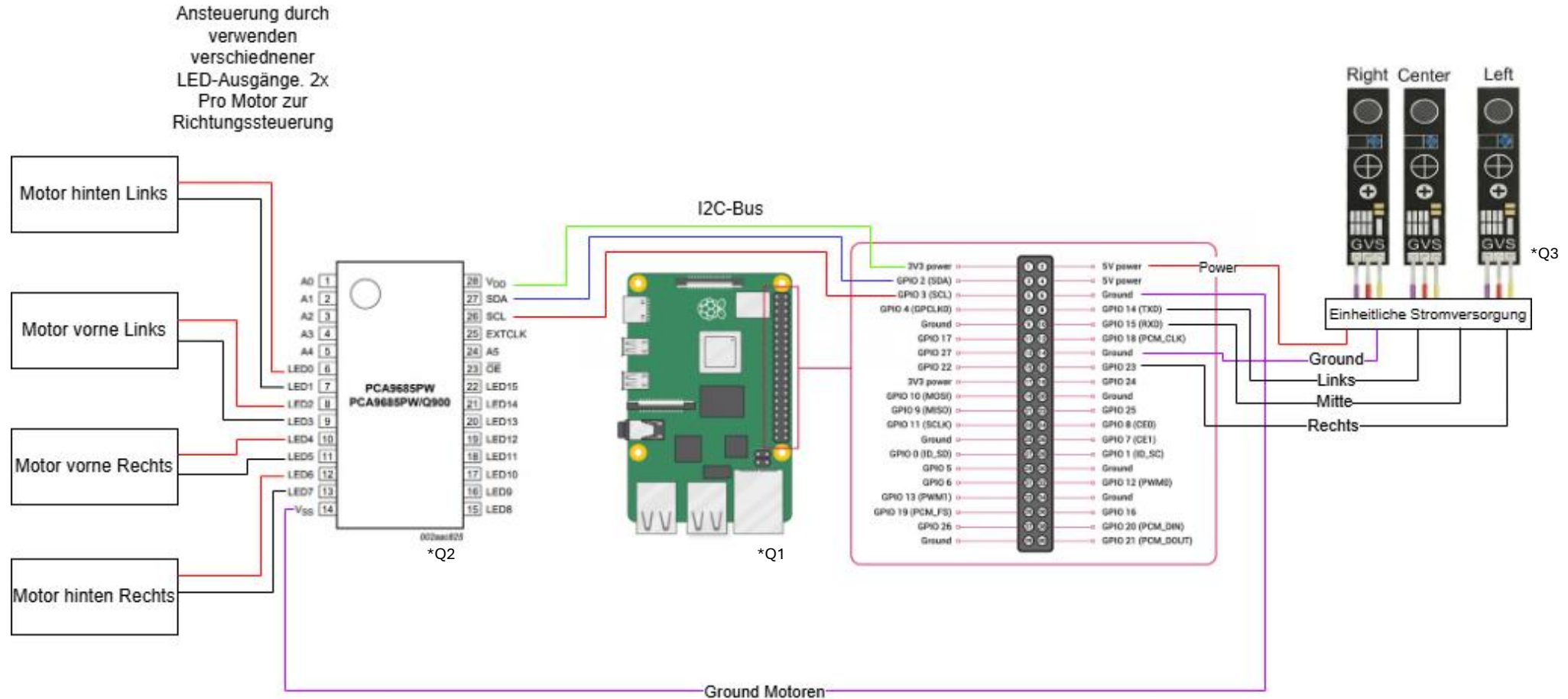
Zu PID:

- Schnelle Reaktion auf (enge) Kurven, da direkteres Ansprechverhalten
- Zuverlässig trotz nur 3 Sensoren und ungenauer Motoren
- Einfacher Verständlich

Zu reinem Bang-Bang:

- Abstufungen in der Stärke der Reaktion
- Gut anpassbar auf Streckenlayout
- Ruhiges Verhalten auf Geraden

Schaltplan des Robocars



Vielen Dank für eure Aufmerksamkeit!

Gibt es noch Fragen?

Bildquellen:

Q1: Bild Raspberry PI (15.05.2026)

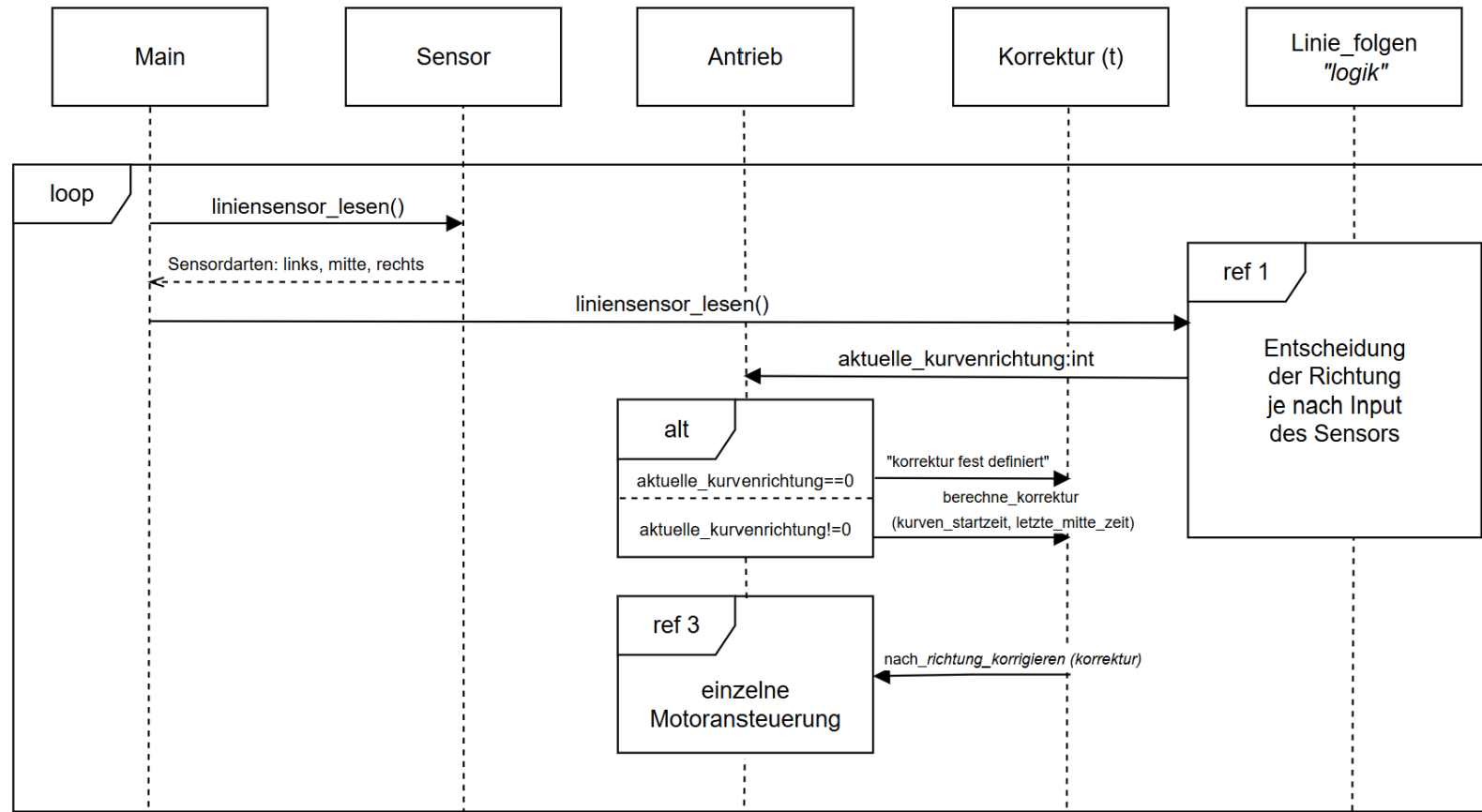
Q2: Bild PCA (15.05.2026)

Q3: Sensor (15.05.2026)

Zusatzfolien für Doku

Oder Fragen

Algorithmus der Regelung



Algorithmus der Regelung

Ref-Frameworks:

